AMRITA VISHWA VIDYAPEETHAM, AMRITAPURI
B. Tech – Computer Science Engineering (Artificial Intelligence)
22AIE401 – Reinforcement Learning

Risk-Aware Adaptive API Rate Limiting Using Reinforcement Learning

Name	Roll No
Avikshith Arvind	AM.SC.U4AIE23025
Ananthakrishnan S	AM.SC.U4AIE23019
Aman Backer	AM.SC.U4AIE23016

Problem Background

The current applications depend significantly on Application Programming Interfaces (APIs) for communicating among clients, services, databases, clouds, and micro services. In cases where an API receives many requests at once from different clients, traffic control becomes one of the requirements for the sustainability of the application.

The traditional methods of controlling the rate of APIs generally apply pre-set thresholds like limiting the number of requests for a client in a certain time interval. Even if Token Bucket, Leaky Bucket, and Sliding Window techniques manage the traffic effectively, they tend to work using predefined or manually set limits.

The problem is that the actual traffic is dynamic, where an increase in requests might mean either legitimate bursts in traffic like simultaneous requests by many clients or an attack. The same limit applied in both cases could restrict legitimate users or even fail in protecting the back-end system.

Moreover, the right rate limit depends on the state of the backend. A state of high CPU usage, higher latency and usage of memory and increasing queue of requests may show that there is a need for reducing the amount of traffic to the server.

Reinforcement Learning is an approach in which decisions are learned through the interactions with the environment. The research project is focused on applying reinforcement learning to adaptive API rate limiting when the agent learns to adapt traffic control policy based on the client behavior and system state**.**

Problem Statement

Most API gateways employ rate limiting methods based on pre-defined rate limits that allow clients to generate a certain number of requests. Even though these techniques are quite effective, rate limiting that doesn't change in response to various conditions is hard to apply to variable traffic patterns, different client actions, or backend system conditions.

For instance, if there is a legitimate client who suddenly generates a burst of requests, the client will be regarded the same way as the one who generates malicious traffic. Moreover, the traffic rate limit won't change even if the backend system suffers from increased CPU utilization or latency.

Thus, it is crucial to have an adaptive traffic management system that would be able to adaptively define the optimal traffic control policies depending on the current condition of the client and backend system.

As a part of the proposed project, an API gateway based on reinforcement learning will observe different parameters like request rate, traffic burstiness, client actions, server load, response latency, and error rate, and take appropriate traffic-management actions.

The system will attempt to balance:

- legitimate request access
- prevention of API abuse
- backend stability
- response latency
- efficient resource utilization
-

Why Reinforcement Learning?

The problem of API traffic management itself could be easily defined as a sequential decision making problem.

At each point in time, the system is aware of the current state of API traffic and the backend environment. On the basis of this state, a decision needs to be made concerning the amount of traffic that can be allowed through.

For example :

Current State —> High traffic —> High CPU —> RL Agent —> Reduce rate limit —> Traffic decreases —> CPU and latency improve —> New State
Unlike traditional rule-based systems, where thresholds are manually defined, an RL agent can learn a policy through interaction with an API traffic environment.

The problem can be represented using:

State:
Current request rate, client risk, CPU utilization, memory utilization, latency, error rate, and current rate limit.

Action:
Increase, maintain, decrease, throttle, or block traffic.

Reward:
Reward the system for serving legitimate requests while penalizing backend overload, abusive requests, excessive latency, and unnecessary throttling.

Literature Review

S. No	Paper	Year
1	**Adaptive Rate Limiting Using Reinforcement Learning to Thwart API Abuse**	2023
2	**TopFull: An Adaptive Top-Down Overload Control for SLO-Oriented Microservices**	2024
3	**Multi-Objective Adaptive Rate Limiting in Microservices Using Deep Reinforcement Learning**	2025
4	**DAGOR: Overload Control for Scalable Microservices**	2018
5	**Breakwater: Overload Control for μs-scale RPCs**	2020
6	**Machine Learning Enhanced API Rate Limiting and Throttling Strategies in High Volume Fintech Systems**	2022

Existing System

Current API management systems commonly use predefined rate-limiting mechanisms such as Token Bucket, Leaky Bucket, Fixed Window, and Sliding Window algorithms.

Cloud platforms and API security platforms have also introduced adaptive mechanisms for detecting abnormal traffic and managing API abuse.
For example, **Cloudflare API Shield** provides adaptive mechanisms for identifying volumetric API abuse based on observed traffic behavior.

Systems such as **Envoy** also provide admission-control mechanisms that can dynamically reject requests based on observed service conditions.

Research systems such as **TopFull** investigate reinforcement-learning-based admission control for microservices and dynamically regulate API traffic according to system conditions.

However, existing systems generally address specific aspects of the problem, such as:

- fixed traffic thresholds
- anomaly detection
- backend overload
- API abuse prevention
- service-level objectives

The proposed project aims to combine **client behavioral information and backend health information** within an RL-based traffic-control framework.

Gap Analysis

The traditional methods for rate limiting APIs are based mainly on preconfigured thresholds; therefore, these methods are rather inflexible in coping with fast changes in the traffic flow. The threshold is optimal in the case of normal workload but may become too strict or too liberal when there is a change in the traffic flow.

Previously, researchers have tried to design an adaptive rate limiter based on reinforcement learning, but there are other algorithms that are designed for detecting API abuses or for microservices overload control. These solutions are usually centered on one particular aspect of the problem. Namely, API abuses detectors are aimed at finding suspicious clients.

Another significant issue concerns separating legitimate traffic from malicious traffic. Having a big number of requests from a client does not automatically imply that the client acts maliciously. Likewise, a traffic increase might be due to a genuine activity rather than being an attack. Thus, request volume may not be enough for making the proper rate limiting decision.

There is also a necessity to take into account the current state of the backend in making the decision about traffic control. The same client traffic will have different impacts based on CPU utilization, memory utilization, response time and other factors. Therefore, there is a need for having an adaptive traffic control technique that takes into account client behavior and the state of the backend.

Proposed System

The proposed project aims to develop a **Risk-Aware Adaptive API Rate Limiting system using Reinforcement Learning**. Unlike conventional API gateways that rely on fixed rate limits, the proposed system will continuously observe incoming API traffic, client behavior, and backend system conditions to determine an appropriate traffic-control policy.

The system will collect information such as request frequency, traffic burstiness, historical client behavior, error rate, suspicious request patterns, CPU utilization, memory utilization, response latency, and the current rate limit. Client behavioral information can be used to estimate a risk score, while backend monitoring provides information about the current health of the service.

These observations will be provided to an RL agent, which will learn to select an appropriate traffic control action. The possible actions may include increasing the rate limit, maintaining the current limit, decreasing the rate limit, throttling the client, or temporarily blocking the client. The agent will receive rewards for successfully serving legitimate traffic and maintaining backend stability, while penalties will be associated with abusive requests, excessive latency, backend overload, and unnecessary throttling of legitimate users.

The proposed system will initially investigate a Q-learning-based approach and evaluate its performance using simulated API workloads. The simulation will include normal traffic, sudden traffic bursts, high-volume legitimate traffic, abusive clients, and backend overload conditions. The proposed approach will then be compared with conventional rate-limiting techniques to study its effectiveness in dynamically managing API traffic.

The overall research direction is therefore to investigate whether combining **client behavioral risk with real-time backend health in an RL-based decision-making framework** can provide more adaptive API traffic management while maintaining service stability and legitimate-user access.

Initial RL Formulation: State, Action, Reward, Policy, Objective

The API rate-limiting problem is formulated as a Markov Decision Process (MDP) $\mathbf{M = (S, A, T, R, \gamma)}$, where the RL agent continuously observes API traffic and backend system conditions and learns an adaptive traffic-control policy.

State (S)

The state represents the current condition of both the API client traffic and the backend system:

$$\mathbf{s_t = (r_t, b_t, c_t, cpu_t, mem_t, l_t, err_t, q_t)}$$

where $\mathbf{r_t}$ is the current request rate, $\mathbf{b_t}$ represents traffic burstiness, $\mathbf{c_t}$ represents the client risk score based on observed behaviour, $\mathbf{cpu_t}$ and $\mathbf{mem_t}$ represent CPU and memory utilization, $\mathbf{l_t}$ is the

current response latency, $\mathbf{err}_t$ is the error rate, and $\mathbf{q}_t$ represents the current request/load condition.

The state therefore captures both **client behaviour and backend health**. Considering only request volume would not be sufficient because a sudden increase in requests may represent either legitimate high-volume activity or abusive traffic. Similarly, the same traffic rate can have different effects depending on the current CPU utilization, memory usage, latency, and backend load.

Action (A)

At each decision step, the RL agent selects a traffic-control action according to the current state.

$\mathbf{a}_t \in$ **{increase, maintain, decrease, throttle, block}**

The actions correspond to:

- **Increase** the current rate limit
- **Maintain** the current rate limit
- **Decrease** the rate limit
- **Throttle** the client
- **Temporarily block** the client

The action set allows the agent to dynamically control API traffic instead of applying one fixed threshold to every client and traffic condition.

Reward (R)

The reward is designed to encourage legitimate API access while maintaining backend stability:

$\mathbf{R(s_t, a_t) = R_legitimate - P_abuse - P_overload - P_latency - P_throttle}$

where the positive component rewards successfully serving legitimate requests, while penalties are applied for abusive requests, backend overload, excessive latency, and unnecessary throttling of legitimate users.

Thus, the agent is encouraged to find a balance between **legitimate request access, API abuse prevention, backend stability, response latency, and efficient resource utilization**.

Policy (π)

The policy π**(als)** represents the probability or learned preference of selecting a particular traffic-control action for a given system state.

In the proposed system, the agent initially uses **Q-learning** to learn the relationship between states and actions. The learned Q-values indicate how beneficial each possible rate-limiting action is for a particular state.

The policy can therefore be represented as:

$\pi(\mathbf{s}_t) = \mathbf{argmax}_a\ \mathbf{Q}(\mathbf{s}_t, \mathbf{a})$

during exploitation, with exploration used during training so that the agent can discover better actions.

The policy allows the system to move from a fixed rule such as "*allow 100 requests per minute*" toward an adaptive decision such as:

High traffic + high CPU + high latency → decrease/throttle

while:

High traffic + low CPU + low latency + low client risk → maintain/increase

This enables the rate limit to adapt to changing traffic and backend conditions.

Objective

The objective of the RL agent is to learn a rate-limiting policy that **maximizes cumulative reward over time**:

$$\pi^* = \text{argmax}_\pi \, E[\Sigma_t \, \gamma^t \, R(s_t, a_t)]$$

where γ is the discount factor.

The learned policy should serve legitimate requests while reducing abusive traffic and preventing unnecessary backend overload. The project will initially investigate a **Q-learning-based approach** and evaluate the learned policy using simulated API workloads containing normal traffic, sudden bursts, high-volume legitimate traffic, abusive clients, and backend overload conditions.

The overall learning process can be represented as:

State → Q-Learning Agent → Action → API Environment → Reward → Updated Q-values → New Policy

This formulation directly supports the project's goal of investigating whether combining **client behavioral risk and real-time backend health** can provide more adaptive API traffic management while maintaining service stability and legitimate-user access.

High-Level Solution Diagram

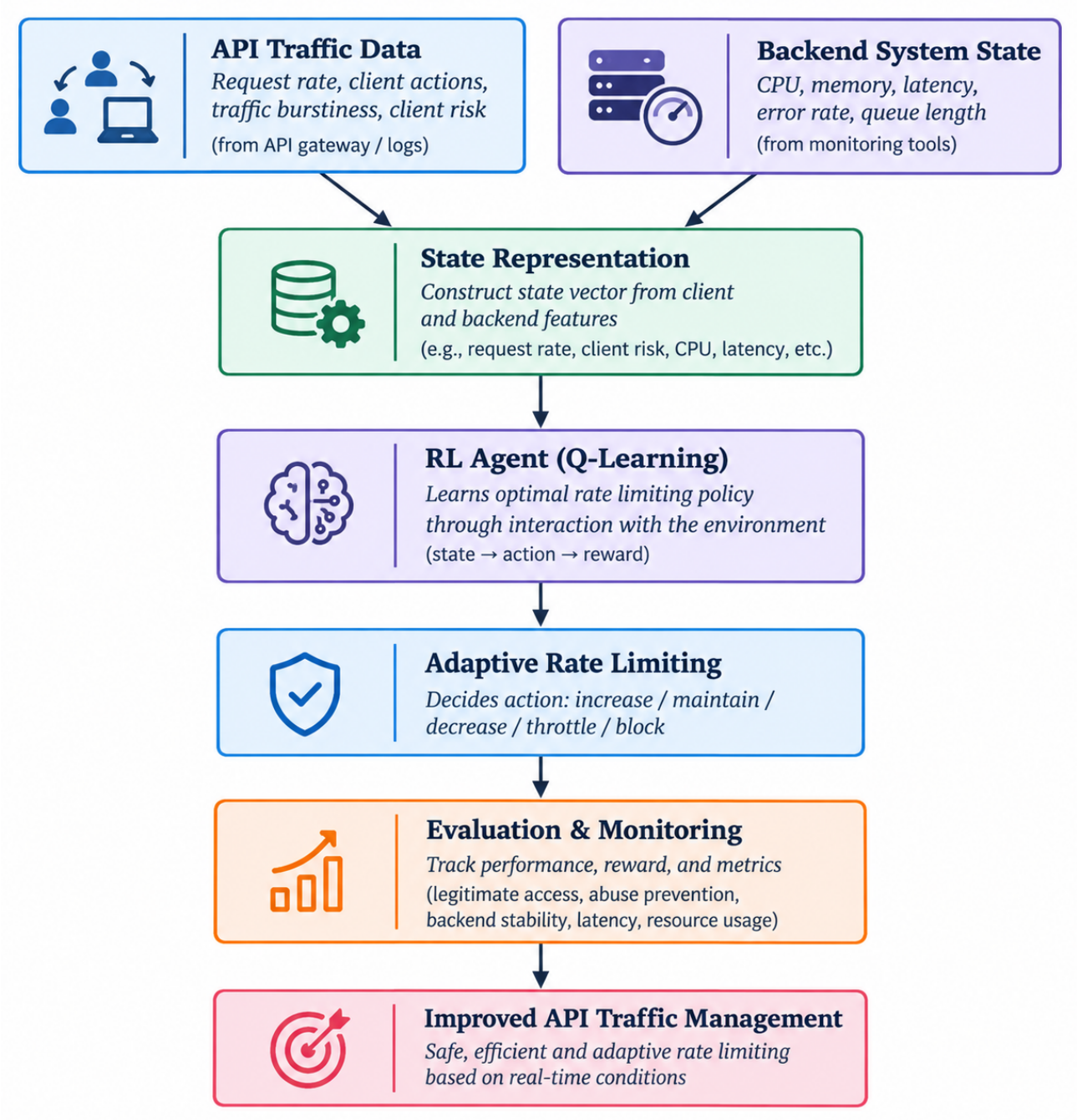


Figure 1. High-Level Solution Architecture

The proposed system models API traffic management as a sequential decision-making problem. The RL agent receives observations from two main sources: **API/client traffic behaviour** and **backend system health**. These observations are combined to form the current state of the environment.

The state is passed to the **Q-learning agent**, which selects one of the available traffic-control actions: increase, maintain, decrease, throttle, or temporarily block the client's rate limit. The selected action changes the traffic admitted to the API and consequently affects backend load, latency, errors, and subsequent observations.

The interaction follows the RL loop:

API Traffic + Backend State → State Representation → Q-Learning Agent → Rate-Limiting Action → API Environment → Reward → Q-value Update → New State

The system is intended to distinguish between legitimate traffic bursts and potentially abusive traffic while also considering the current condition of the backend. Thus, a high request rate does not automatically result in aggressive throttling; the decision also depends on client risk and backend health.

The proposed high-level pipeline consists of:

1. **API Traffic Data** — request rate, burstiness, client behaviour, suspicious patterns and historical behaviour.
2. **Backend System State** — CPU utilization, memory utilization, response latency, error rate and current load.
3. **State Representation** — combines client-side and backend features into the RL state.
4. **Q-Learning Agent** — learns the relationship between system states and traffic-control actions.
5. **Adaptive Rate Limiting** — applies the selected action to increase, maintain, decrease, throttle or block traffic.
6. **Reward and Monitoring** — evaluates legitimate access, abuse prevention, latency, backend stability and unnecessary throttling.
7. **Policy Update** — updates the learned Q-values based on the observed reward.

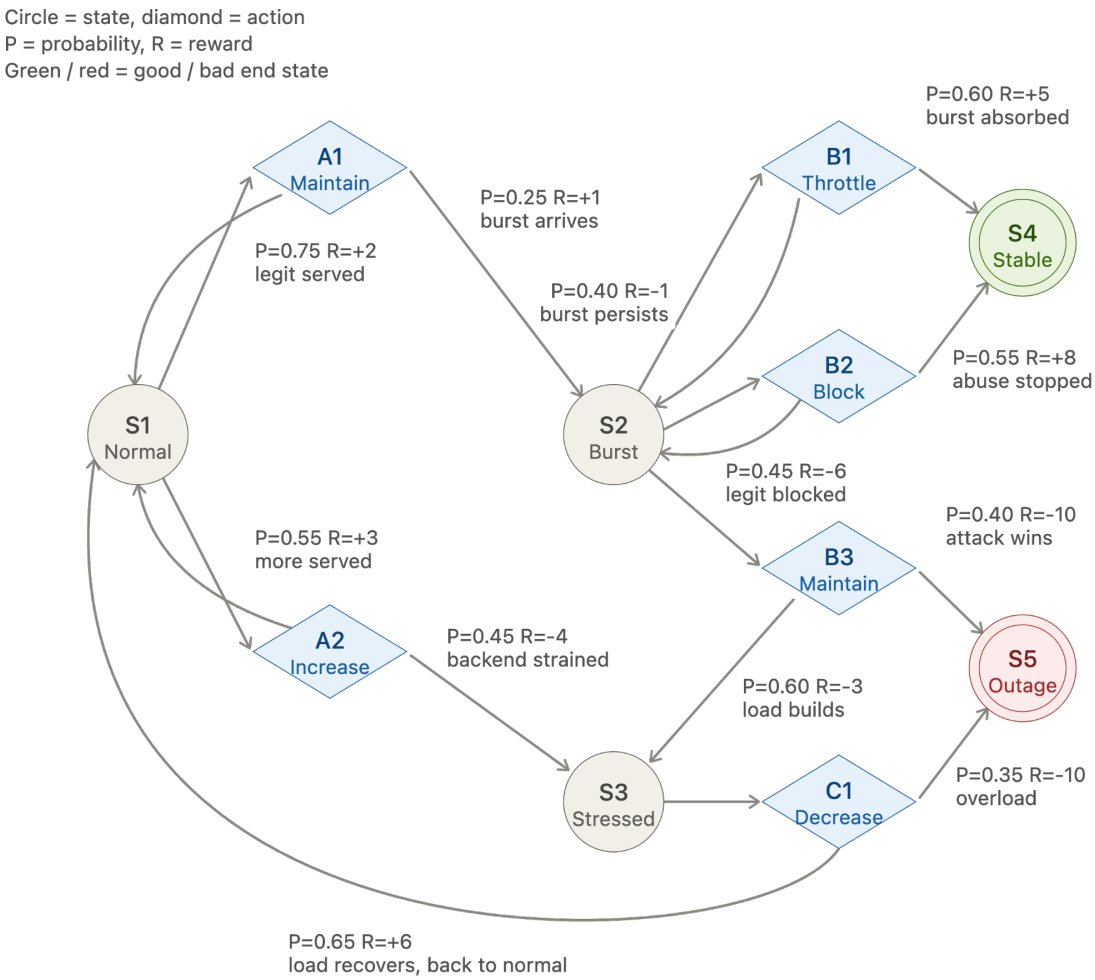


Figure 2. Illustrative RL Decision Loop

The diagram below represents the RL mechanics of the proposed system. A particular API traffic condition produces a state, the agent selects a rate-control action, the API environment changes accordingly, and a reward is generated based on the resulting system condition. The actual state space consists of combinations of client behaviour and backend conditions, while the learned Q-values are obtained through interaction with the simulated API environment.

11. Dataset / Simulation Plan

• **API Traffic Simulation:** Since the proposed project is initially evaluated using simulated API workloads, synthetic API request streams will be generated to represent different client and traffic behaviours. The simulation will include request frequency, burstiness, client behaviour and suspicious request patterns.

• **Backend Simulation:** The backend environment will simulate system-health variables such as CPU utilization, memory utilization, response latency, error rate and request/load conditions. This allows the RL agent to observe how the same traffic pattern can have different effects under different backend conditions.

• **Traffic Scenarios:** The simulation will contain several workload categories:

- Normal API traffic
- Sudden traffic bursts
- High-volume legitimate traffic
- Abusive clients
- Backend overload
- Mixed legitimate and abusive traffic

• **Client Risk:** A client risk representation will be derived from observed behavioural characteristics such as request frequency, burstiness, historical behaviour and suspicious request patterns. The risk information will be provided as part of the RL state rather than using request volume alone.

• **Training:** The Q-learning agent will interact repeatedly with the simulated environment. At each step, it observes the current state, selects an action, receives a reward, and updates its Q-value.

• **Evaluation:** After training, the learned policy will be tested on simulated traffic conditions and compared against conventional rate-limiting approaches. The project specifically proposes comparing the RL approach with conventional rate-limiting techniques to study its effectiveness in dynamically managing API traffic.

Evaluation Metrics and Baseline

Metrics

Metric	What it measures
Legitimate Request Acceptance Rate	Percentage of legitimate API requests successfully served without unnecessary restriction.
Abuse Blocking / Throttling Rate	How effectively abusive or suspicious traffic is restricted.
Average Response Latency	Average time required to respond to API requests under different traffic conditions.
Backend CPU Utilization	CPU resource consumption during API traffic.
Memory Utilization	Memory pressure caused by incoming API traffic.
Error Rate	Percentage of API requests resulting in errors or failures.

Backend Overload Events	Frequency of situations where the backend reaches an overloaded condition.
Unnecessary Throttling Rate	Percentage of legitimate requests that are incorrectly throttled or blocked.
Cumulative Reward	Overall reward obtained by the RL agent during an episode, representing the quality of its learned decisions.

These metrics reflect the project's stated objective of balancing **legitimate request access, prevention of API abuse, backend stability, response latency and efficient resource utilization**.

Baselines

The baselines are selected from conventional API rate-limiting approaches so that the adaptive RL approach can be evaluated against fixed or rule-based traffic-control mechanisms.

Baseline	Description
Token Bucket	Allows requests according to a predefined token generation and consumption rate.
Leaky Bucket	Controls the outgoing request rate using a predefined processing rate.
Fixed Window	Limits the number of requests allowed within a fixed time interval.
Sliding Window	Controls requests using a continuously moving time window.

These techniques are identified in the project as commonly used predefined rate-limiting mechanisms.

The proposed **Q-learning-based adaptive rate limiter** will be evaluated against these approaches under normal traffic, bursts, legitimate high-volume traffic, abusive clients and backend overload conditions.

Expected Novelty

• **Joint consideration of client risk and backend health:** Unlike a rate limiter based primarily on a

fixed request threshold, the proposed framework combines client behavioural information with real-time backend conditions. The project specifically identifies this combination as the main research direction.

• **Adaptive traffic-control policy:** Instead of maintaining a manually configured rate limit under changing conditions, the Q-learning agent learns when to increase, maintain, decrease, throttle or block traffic based on the observed state.

• **Risk-aware decision making:** A high request volume is not treated automatically as malicious. The framework incorporates client behaviour and risk information so that legitimate traffic bursts can be distinguished from potentially abusive behaviour.

• **Backend-aware rate limiting:** The decision is also influenced by CPU utilization, memory utilization, latency, error rate and system load. Therefore, the same client traffic can result in different actions depending on the current backend condition.

• **Multi-objective traffic management:** The reward design simultaneously considers legitimate access, abuse prevention, latency, backend overload and unnecessary throttling rather than optimizing only a single traffic-control objective.

• **Reproducible simulation-based evaluation:** The initial implementation uses simulated API workloads containing normal traffic, legitimate bursts, abusive clients and backend overload conditions, allowing the learned policy to be evaluated systematically against conventional rate-limiting techniques.

The overall expected contribution is therefore a **risk-aware, backend-aware adaptive API rate-limiting framework using Q-learning**, with the goal of improving traffic management while maintaining legitimate-user access and backend stability.

References

The current project document provides the following literature-review references by title and year. Since the uploaded document does not contain complete author, venue, DOI, or URL information for these entries, those details should not be added without verification.

[1] *Adaptive Rate Limiting Using Reinforcement Learning to Thwart API Abuse*, 2023.

[2] *TopFull: An Adaptive Top-Down Overload Control for SLO-Oriented Microservices*, 2024.

[3] *Multi-Objective Adaptive Rate Limiting in Microservices Using Deep Reinforcement Learning*, 2025.

[4] *DAGOR: Overload Control for Scalable Microservices*, 2018.

[5] *Breakwater: Overload Control for µs-scale RPCs*, 2020.

[6] *Machine Learning Enhanced API Rate Limiting and Throttling Strategies in High Volume Fintech Systems*, 2022.

The project also discusses **Cloudflare API Shield** as an example of adaptive API-abuse detection

and **Envoy** as an example of admission-control mechanisms that can reject requests according to service conditions.